# hard-label-dnn-extraction 实验产物与结果文件存储指引

本文档详细说明本项目各项实验执行后**生成的结果文件、目录层级、具体数据格式及其在本地与服务器上的存放位置。**

## 目录


1. 产物存储总览与速查表
2. 实验一：端到端模型完全窃取产物（`full_end_to_end/out/`）
3. 实验二：标准 ReLU 独立黑盒符号恢复产物（`sign_recovery/results/blackbox/`）
4. 实验三：CIFAR-10 白盒符号恢复产物（`sign_recovery/results/whitebox/`）
5. 实验四：Leaky ReLU 符号恢复运行机制（`sign_recovery_leaky`）
6. 技术评估报告与 PDF 文件清单


## 1. 产物存储总览与速查表

| 实验模块 | 产物目录相对路径 | 核心产物文件 | 主要数据内容 |
|---|---|---|---|
| **端到端完全窃取** | `full_end_to_end/out/` | `net.npz`，`stats.json`，`layer*.npy` | 窃取的完整神经网络权重、各阶段查询与耗时统计、决策面双点 |
| **黑盒符号恢复** | `sign_recovery/results/blackbox/` | `df.pkl`，`df.csv`，`df.md` | 每个神经元独立统计检验数据、投票记录与置信度 |
| **白盒符号恢复** | `sign_recovery/results/whitebox/` | `df.pkl`，`df.csv`，`df.md` | CIFAR-10 深度网络各神经元的移动速度比与置信度 |
| **Leaky 符号恢复** | 内存与标准输出 (stdout) | 无落盘大文件（即时验证） | 终端即时输出各层 32/32 正确判定结果 |

## 2. 实验一：端到端模型完全窃取产物（`full_end_to_end/out/`）

这是本论文最核心的产物目录。只要执行了 `python run.py [WORKERS]`，所有中间与最终结果均保存在 `out/` 中。

### 2.1 目录结构与具体文件

```
full_end_to_end/out/
├── net.npz                  # 【最核心产物】被窃取的完整神经网络权重压缩包
├── stats.json               # 全流程每个阶段的耗时与 Oracle 查询次数统计
├── layer0.npy               # 第 1 隐藏层恢复的 [权重矩阵 | 偏置向量]，shape=(32, 33)
├── layer0_refined.npy       # 第 1 隐藏层经过 Kink 微分精修后的高精度行向量
├── layer0_rows.npy          # 第 1 隐藏层初次求解超平面得到的行向量
├── layer1.npy               # 第 2 隐藏层最终参数文件
├── layer1_refined.npy       # 第 2 隐藏层精修文件
├── layer1_rows.npy          # 第 2 隐藏层初次求解文件
├── layer2.npy               # 第 3 隐藏层最终参数文件
├── layer2_refined.npy       # 第 3 隐藏层精修文件
├── layer2_rows.npy          # 第 3 隐藏层初次求解文件
├── duals/                   # 边界漫步采集到的全部双点（拐点）数据文件夹
│   ├── walk1.p              # 第 1 条路径收集的双点 (pickle 格式)
│   └── ...                  # 共 8 个 walk 文件 (walk1.p ~ walk8.p)
└── logs/                    # 全部 96 个神经元的符号判定详细日志文件夹
    ├── signs_layer0_neuron0.log
    └── ...                  # 包含 signs_layer0_neuron0.log 到 signs_layer2_neuron31.log
```

### 2.2 核心文件内部格式解析

- **`net.npz`**：NumPy 字典打包文件，可用 `np.load("net.npz")` 读取：
- `net["L0"]`：第 1 隐藏层权重与偏置（包含正负符号）；
- `net["L1"]`：第 2 隐藏层权重与偏置；
- `net["L2"]`：第 3 隐藏层权重与偏置；
- `net["R"]`：$32 \times 10$ 输出分类头矩阵。
- **`stats.json`**：记录了 `walk`、`layer0/cluster`、`layer0/solve`、`layer0/refine`、`layer0/signs`、`head` 等各阶段的具体 `queries` 与 `seconds`。

### 2.3 物理存放路径对照

- **本地端运行生成位置**：`C:\Users\rsty\Desktop\GI\hard-label-dnn-extraction\full_end_to_end\out\`
- **服务器端运行原始位置**：`~/project/HZR/hard-label-dnn-extraction/full_end_to_end/out/`
- **从服务器传回本地的副本**：`C:\Users\rsty\Desktop\GI\out\`

---

## 3. 实验二：标准 ReLU 独立黑盒符号恢复产物（`sign_recovery/results/blackbox/`）

### 3.1 目录结构

```
sign_recovery/results/blackbox/
└── unitary_32_32x3_10_float64/
    ├── layerID_1/
    │   ├── neuronID_0/
    │   │   ├── df.pkl       # Pandas DataFrame 二进制文件（create_tables.py 读取来源）
    │   │   ├── df.csv       # CSV 格式的每次微扰实验记录表
    │   │   └── df.md        # 单神经元 Markdown 结果卡片
    │   ├── neuronID_1/
    │   └── ...
    ├── layerID_2/
    └── layerID_3/
```

### 3.2 字段说明

- `df.csv` / `df.pkl` **内部记录**：每次实验的微扰位移、正负侧翻转判定（`Vote dOFF>dON`）、子实验耗时（`subpoint_time_seconds`）等。
- **制表工具**：在 `sign_recovery` 目录下执行 `python create_tables.py`，程序会自动扫描读取上述所有 `df.pkl` 并生成汇总大表。

### 3.3 物理存放路径对照

- **本地端路径**：`C:\Users\rsty\Desktop\GI\hard-label-dnn-extraction\sign_recovery\results\blackbox\`
- **服务器端路径**：`~/project/HZR/hard-label-dnn-extraction/sign_recovery/results/blackbox\`

---

## 4. 实验三：CIFAR-10 白盒符号恢复产物（`sign_recovery/results/whitebox/`）

### 4.1 目录结构

```
sign_recovery/results/whitebox/
└── cifar10_3x256_64_10_float64/
    └── layerID_1/
        ├── neuronID_0/
        │   ├── df.pkl      # 包含速度比 ||vON||/||vOFF|| 与置信度数据
        │   ├── df.csv
        │   └── df.md
        ├── neuronID_1/
        └── ... (neuronID_0 至 neuronID_15)
```

### 4.2 物理存放路径

- **服务器端路径**：`~/project/HZR/hard-label-dnn-extraction/sign_recovery/results/whitebox/`

---

## 5. 实验四：Leaky ReLU 符号恢复运行机制（`sign_recovery_leaky`）

- **数据保存特点**：该算法属于**轻量级在线验证算法**，通过观察激活点两侧梯度跳变的不对称性，每层仅需 ~104 个采样即可 100% 收敛。
- **输出方式**：程序收敛后直接在内存中将符号乘入 `weights`，核心结果直接实时输出在终端控制台（`Layer: 1, Correct signs: 32/32`），**不会在磁盘中创建额外的冗余大文件。**

---

## 6. 技术评估报告与 PDF 文件清单

基于上述底层数据，系统已自动整理汇总出两套完整的技术报告（位于 `GI/` 根目录及项目子目录下）：

### 1. 本地 CPU 全流程运行报告

- 📝 **Markdown 格式**：`C:\Users\rsty\Desktop\GI\CPU_EXPERIMENT_RESULTS.md`
- 📄 **PDF 格式**：`C:\Users\rsty\Desktop\GI\CPU_EXPERIMENT_RESULTS.pdf`

### 2. 服务器端 (RTX 4090 / Linux) 汇总报告

- 📝 **Markdown 格式**：`C:\Users\rsty\Desktop\GI\SERVER_EXPERIMENT_RESULTS.md`
- 📄 **PDF 格式**：`C:\Users\rsty\Desktop\GI\SERVER_EXPERIMENT_RESULTS.pdf`